E280 Arbitrum Expansion

Security Review

January 31, 2026 - February 4, 2026

Contents

1. About SBSecurity 3
2. Disclaimer 3
3. Risk classification 3
4. Executive Summary 4
5. Findings 5

High Severity 6

[H-01] Claimed esGMX Permanently Locked in Vault 6

Low severity 9

[L-01] Missing UniV3 path validation in BuyBurn 9
[I-01] Duplicate Fee Calculation in deposit 10

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **DudeGuy420/E280-ARB-Expansion** repository was conducted by **SBSecurity**, who collectively identified 3 issues in total.

Overview

Project	E280 Arbitrum Expansion
Commit Hash	2bb7f405cca2061f40ca1088648fa006c25bbb26
Resolution	8b8e734abcaa92a2714ce4dd2bc8e90da462461d
Timeline	January 31, 2026 - February 4, 2026

Scope

E280BuyBurn.sol, E280GmxStake.sol, E280LPManager.sol, E280TaxDistributor.sol, WhitelistRegistry.sol

Post Audit Condition

The codebase is well-structured and adheres to strong development practices. No Critical severity issues were found. Additional audits would be a bonus, but are not mandatory.

5. Findings

Issues Found

Critical Risk	0
High Risk	1
Medium Risk	0
Low/Info Risk	2

ID	Title	Severity	Resolution
[H-01]	Claimed esGMX Permanently Locked in Vault	High	Fixed
[L-01]	Missing UniV3 path validation in BuyBurn	Low	Fixed
[I-01]	Duplicate Fee Calculation in deposit	Info	Fixed

High Severity

[H-01] Claimed `esGMX` Permanently Locked in Vault

Severity: High Risk

Description: Every call to `_claimRewards` - triggered on every deposit, mint, withdraw, and redeem via the `autoClaimRewards` modifier - claims esGMX into the vault with `shouldClaimEsGmx: true` but does not re-stake it (`shouldStakeEsGmx: false`). esGMX uses `inPrivateTransferMode` in the GMX protocol, restricting transfers to whitelisted handlers only. Since the vault is not a handler on the esGMX token, `rescueERC20` reverts on `safeTransfer`, and the vault has no vesting infrastructure to convert esGMX to GMX. The esGMX accumulates indefinitely with no recovery path.

```
function _claimRewards() internal returns (uint256 claimedGmx) {
    if (isSunset) return 0;
    IERC20 gmx = IERC20(asset());
    uint256 balanceBefore = gmx.balanceOf(address(this));

    REWARD_ROUTER.handleRewards(
        true,  // shouldClaimGmx
        false, // shouldStakeGmx
        true,  // shouldClaimEsGmx    ← claimed to vault
        false, // shouldStakeEsGmx    ← never re-staked
        false, // shouldStakeMultiplierPoints
        true,  // shouldClaimWeth
        false  // shouldConvertWethToEth
    );

    claimedGmx = gmx.balanceOf(address(this)) - balanceBefore;
    // ... only GMX is allocated; esGMX sits idle in the vault
}
```

The stakedGmxTracker (which esGMX is claimed through) enforces `inPrivateTransferMode` via `_validateHandler`:

```
function _transfer(address _sender, address _recipient, uint256 _amount) private {
    require(_sender != address(0), "RewardTracker: transfer from the zero address");
    require(_recipient != address(0), "RewardTracker: transfer to the zero address");

    if (inPrivateTransferMode) { _validateHandler(); }

    balances[_sender] = balances[_sender].sub(_amount, "RewardTracker: transfer amount exceeds balance");
    balances[_recipient] = balances[_recipient].add(_amount);

    emit Transfer(_sender, _recipient,_amount);
}

function _validateHandler() private view {
    require(isHandler[msg.sender], "RewardTracker: forbidden");
}
```

When `rescueERC20` calls `safeTransfer`, `msg.sender` is the vault - which is not a registered handler:

```
function rescueERC20(address token, uint256 amount, address to) external onlyOwner {
    if (token == address(0) || to == address(0)) revert ZeroAddress();
    if (token == address(asset())) revert Prohibited();
    if (token == E280) revert Prohibited();
    IERC20(token).safeTransfer(to, amount); // reverts — vault is not a handler
}
```

Impact: All esGMX claimed over the vault's lifetime is permanently locked. esGMX converts 1:1 to GMX when vested and earns the same rewards as regular GMX when staked, so every claim cycle removes real value from the system that should compound into the vault's staked position or flow to genesis/buyBurn. The loss grows with every user interaction (deposit, mint, withdraw, redeem) since each triggers `autoClaimRewards`.

Proof of Concept:

Fork test demonstrating esGMX is locked in the vault and `rescueERC20` reverts:

1. Paste the test (or here)

```
function test_esGmxPermanentlyLocked() public {
    // 1. User deposits GMX — vault stakes it via RewardRouter
    deal(GMX, user, 1000 ether);

    vm.startPrank(user);
    IERC20(GMX).approve(address(vault), 1000 ether);
    vault.deposit(1000 ether, user);
    vm.stopPrank();

    // 2. Simulate esGMX accrual from handleRewards(shouldClaimEsGmx: true).
    //    esGMX emission rate from stakedGmxTracker is currently 0 on mainnet
    //    (GMX DAO migrated to direct GMX buyback), so we deal esGMX directly
    //    to the vault to replicate the state after a claim.
    deal(ES_GMX, address(vault), 100 ether);

    uint256 lockedEsGmx = IERC20(ES_GMX).balanceOf(address(vault));
    assertEq(lockedEsGmx, 100 ether);
    emit log_named_decimal_uint("esGMX locked in vault", lockedEsGmx, 18);

    // 3. rescueERC20 reverts — esGMX has inPrivateTransferMode enabled,
    //    vault is not a whitelisted handler on the esGMX token
    vm.prank(owner);
    vm.expectRevert("RewardTracker: forbidden");
    vault.rescueERC20(ES_GMX, lockedEsGmx, owner);
}
```

2. Execute with `ARBITRUM_RPC_URL=<ARB_RPC_ID> forge test --match-test test_esGmxPermanentlyLocked -vvv`

```
[PASS] test_esGmxPermanentlyLocked() (gas: 1272988) Logs: esGMX locked in vault: 100.000000000000000000
```

Recommendation: Set `shouldStakeEsGmx` to `true` in `handleRewards` so esGMX is automatically re-staked. This earns the same rewards as regular GMX and avoids the transfer restriction entirely since the tokens never leave the GMX staking system.

```
REWARD_ROUTER.handleRewards(
    true,  // shouldClaimGmx
    false, // shouldStakeGmx
    true,  // shouldClaimEsGmx
-   false, // shouldStakeEsGmx
+   true,  // shouldStakeEsGmx - re-stake to earn compound rewards
    false, // shouldStakeMultiplierPoints
    true,  // shouldClaimWeth
    false  // shouldConvertWethToEth
);
```

Additionally, `_distributeProtocolBalance` should call `REWARD_ROUTER.unstakeEsGmx` before sunset completes, so any re-staked esGMX is withdrawn from the tracker chain and not left permanently staked in the GMX system.

Resolution: Fixed

Low severity

[L-01] Missing UniV3 path validation in BuyBurn

Severity: Low Risk

Description: `addUniswapV3MultihopToken` at E280BuyBurn.sol does not validate the `path` parameter.

```
function addUniswapV3MultihopToken(
    address token,
    bytes memory path,
    uint256 capPerSwap,
    uint16 incentiveBps,
    uint32 interval
) external onlyOwner {
    if (token == address(0)) revert ZeroAddress();
    if (!_enabledTokens.add(token)) revert DuplicateSwapToken();
    swapSettings[token] = SwapTokenSettings(SwapTypes.UNI_V3, incentiveBps, interval, capPerSwap);
    multihopSwapOptionsV3[token] = path;
    isMultihopSwap[token] = true;
    emit SettingsUpdate();
}
```

In contrast, `addUniswapV2Token` at line 187 validates that the path has at least 2 elements and starts with the correct `token` address. The V3 multihop variant stores the raw `bytes path` without checking its length or whether it encodes the correct starting token.

Impact: An owner misconfiguration (empty path or mismatched starting token) would go undetected at registration time. The token would appear enabled but all `swapToken` calls would revert at the Uniswap V3 router, requiring the owner to disable and re-add the token with a corrected path.

Proof of Concept:

1. Owner calls `addUniswapV3MultihopToken` with an empty or mismatched `path` - transaction succeeds.

2. Whitelisted caller invokes `swapToken` for the token - reverts in `ISwapRouter.exactInput` due to the invalid path.

Recommendation: Validate that `path.length >= 43` (minimum V3 encoded path: 20-byte address + 3-byte fee + 20-byte address) and that the first 20 bytes decode to `token`, consistent with the validation pattern in `addUniswapV2Token`.

Resolution: Fixed

[I-01] Duplicate Fee Calculation in deposit

Severity: Information Risk

Description: In `deposit`, the fee is calculated twice with the same inputs. First, `previewDeposit(assets)` is called at line 152, which internally calls `_feeFromGross(assets, DEPOSIT_FEE_BPS)`. Then `_feeFromGross(assets, DEPOSIT_FEE_BPS)` is called again directly at line 160 with the same arguments.

```
function deposit(uint256 assets, address receiver)
  public
  override
  autoClaimRewards
  nonReentrant
  returns (uint256 shares)
{
  ...

  shares = previewDeposit(assets); // @audit _feeFromGross called here (inside previewDeposit)
  if (shares == 0) revert Prohibited();

  IERC20(asset()).safeTransferFrom(msg.sender, address(this), assets);

  uint256 fee = _feeFromGross(assets, DEPOSIT_FEE_BPS); // @audit duplicate call
  protocolBalance += fee;

  ...
}
```

Impact: Wasted gas on every `deposit` call. The duplicate `_feeFromGross` performs an unnecessary `mulDiv` operation each time a user deposits.

Recommendation: Compute the fee once and reuse it:

```
function deposit(uint256 assets, address receiver)
     public
     override
     autoClaimRewards
     nonReentrant
     returns (uint256 shares)
 {
     if (isSunset) revert ContractSunset();
     if (assets == 0) revert Prohibited();

-    shares = previewDeposit(assets);
+    uint256 fee = _feeFromGross(assets, DEPOSIT_FEE_BPS);
+    uint256 netAssets = assets - fee;
+    shares = super.previewDeposit(netAssets);
     if (shares == 0) revert Prohibited();

     IERC20(asset()).safeTransferFrom(msg.sender, address(this), assets);

-    uint256 fee = _feeFromGross(assets, DEPOSIT_FEE_BPS);
     protocolBalance += fee;
     _mint(receiver, shares);
     emit Deposit(msg.sender, receiver, assets, shares);
     _stakeIdle();
 }
```

Resolution: Fixed